Sonora — Cybersecurity

Parsing de PII, taint analysis, tradução via IA e dashboard Risco Cidadão

Documentação de entrega — Global Solutions CitizenTech
FIAP · Turma 3ESPV-2026 · Disciplina: Cybersecurity

Prof. Vitor Lasse

github.com/peguidotte/sonora/tree/main/cybersecurity

1. O que foi pedido

A disciplina de Cybersecurity, na Global Solutions (PDF CitizenTech, pp. 28–29), pede quatro coisas: identificar dados sensíveis (PII) em qualquer entrada, rastrear esse dado até onde ele pode vazar, traduzir falhas de segurança em linguagem simples usando IA, e manter um painel que mostra esse risco ao cidadão.

O Prof. Vitor Lasse esclareceu, no documento “Explicação: Global Solutions” (Teams da turma), que não existe uma base pronta para evoluir: cada grupo constrói o motor do zero, na linguagem que preferir. Os quatro pontos pedidos são:

- Mapear PII: identificar CPF, RG, dados de saúde e senhas em qualquer dado que entra.
- Taint analysis: seguir o dado da entrada até a saída, provando que PII não vaza em log, erro ou resposta sem autenticação.
- Tradução via IA: transformar um bloqueio de segurança numa explicação simples para o cidadão.
- Dashboard Risco Cidadão: um indicador visual alimentado por esses eventos, com o cálculo explicado.

O professor pediu dois materiais separados: este PDF, com a documentação e os prints de código/evidência, e o código-fonte à parte. Construímos isso como um serviço separado, standalone, com exemplos sintéticos — sem mexer no código real de Java, C# ou Mobile.

2. Como o serviço funciona

O serviço recebe um payload qualquer (um JSON) e devolve a versão sanitizada dele, junto com o que foi encontrado. A ideia é simples: uma aplicação manda o dado bruto antes de gravar ou expor algo, e usa a resposta daqui.

Fluxo de uma requisição

```
payload recebido
   -> scanPayload()            [pii]      detecta cada campo sensível e sua categoria
   -> para cada campo sensível:
        taint(valor, fonte)    [taint]    marca o valor como contaminado
        safeLog(valor cru)     [taint]    sink BLOQUEIA (TaintBlockedError)
        translateDiagnostic()  [llm]       traduz o bloqueio para o cidadão
        sanitize(valor)        [taint]    aplica a máscara do motor de PII
        safeLog(valor seguro)  [taint]    sink PERMITE (já sanitizado)
        translateDiagnostic()  [llm]       traduz a sanitização bem-sucedida
   -> calculateRisk(todos os eventos)     [risk]     recalcula o farol Risco Cidadão
   -> dashboard HTML                      [dashboard] mostra farol + tabela de eventos
```

Cada campo sensível vira dois eventos: um mostrando a tentativa bloqueada, com o dado cru, e outro mostrando a tentativa permitida, já com o dado mascarado. Dá para comparar os dois lados lado a lado.

Endpoints

- POST /api/cybersecurity/v1/analyze — recebe { source, payload }, roda o fluxo acima e devolve o que foi encontrado, o payload sanitizado, os eventos novos e o risco atualizado.
- GET /api/cybersecurity/v1/events e /risk — consultam o estado acumulado.
- POST /api/cybersecurity/v1/reset — limpa o log.
- GET /health/ready, /health/live — checagem básica, mesma convenção dos outros serviços do projeto.
- GET /dashboard, POST /dashboard/run-custom, POST /dashboard/reset — o painel interativo (seção 8).

3. Arquitetura e organização do código

O serviço é dividido em módulos pequenos, cada um cuidando de uma parte do problema:

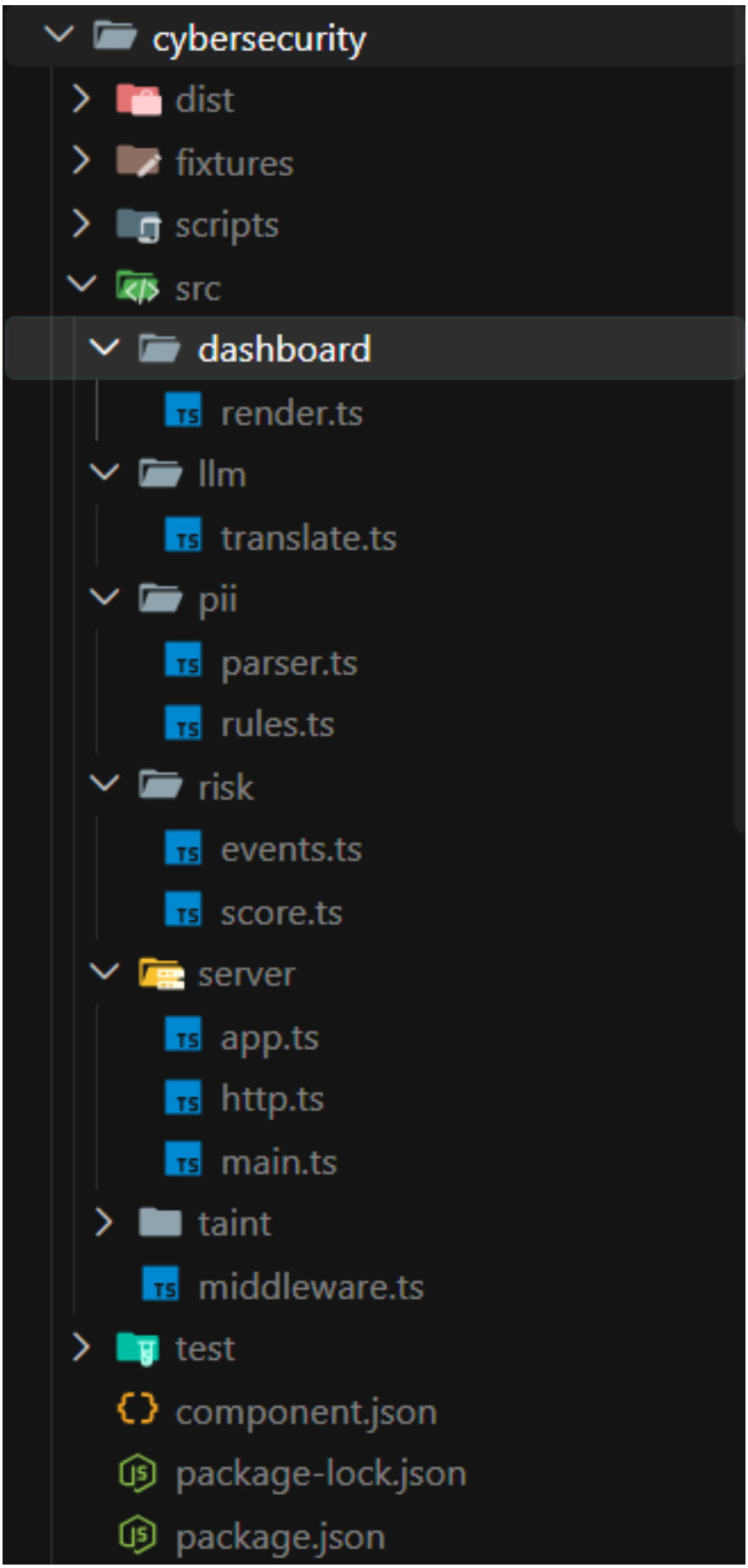


Figura 1 — Estrutura de pastas do serviço (src/)

- src/pii/ — regras de detecção (rules.ts) e a varredura recursiva do payload (parser.ts).

- src/taint/ — o tipo Tainted<T> e os sinks guardados (taint.ts, sinks.ts).
- src/llm/ — tradução do diagnóstico sanitizado, com fallback (translate.ts).
- src/risk/ — log de eventos em memória e cálculo do farol (events.ts, score.ts).
- src/dashboard/ — o HTML do painel estático e do painel interativo (render.ts).
- src/server/ — servidor HTTP sem framework (http.ts, app.ts, main.ts) e o middleware.ts que liga tudo.
- fixtures/requests/ — exemplos sintéticos usados pelos testes, pela demo e pelo painel.
- test/ — 39 testes automatizados (node:test), um arquivo por módulo mais um para o servidor inteiro.

4. Decisões técnicas

- Node.js e TypeScript puro, sem framework HTTP — só node:http, do mesmo jeito que o mock_server.py que já existe no projeto. Menos dependência para auditar.
- Um endpoint só (POST /analyze) em vez de quatro: as etapas são sequenciais — pii acha o dado, taint decide se ele passa, llm traduz, risk soma. Separar em rotas só jogaria essa ordem para fora, sem ganho real.
- Sem chave de IA configurada, a tradução usa um texto fixo e avisa isso na resposta. Com a chave, chama a Anthropic de verdade.
- O LLM nunca recebe o valor bruto — só o tipo de dado, o sink e a fonte do evento.
- Um valor já mascarado não é marcado de novo, mesmo que a chave do campo pareça PII. Sem isso, reenviar um dado já tratado geraria um alarme falso — é a diferença entre os exemplos “vazado” e “mascarado” no painel.
- O log de eventos vive na memória do processo e reinicia junto com ele — serve para auditoria local, não para produção.
- Sem autenticação própria: é uma ferramenta de auditoria da disciplina, não uma rota do produto Sonora.

5. Parsing de PII

O motor (rules.ts e parser.ts) varre qualquer JSON, incluindo objetos aninhados e listas, e aplica quatro regras:

- CPF: formato 000.000.000-00 (ou 11 dígitos), com checksum de verdade — um telefone de 11 dígitos não vira CPF.
- RG: formato 00.000.000-0.
- Saúde: pela chave do campo (ex.: healthNotes) ou por termos como laudo, diagnóstico, prontuário.
- Credencial: chave bate com senha/password/token/apiKey/pin e o valor não está vazio.

Cada regra tem um caso que dispara e um que não dispara: “saudação” não vira “saúde”, uma credencial vazia não conta, e um valor já mascarado não é marcado de novo mesmo com a chave sugerindo PII. O motor devolve a categoria, o caminho do campo e o valor mascarado — nunca o valor original.

O vídeo (seção 9) mostra isso rodando: os exemplos de usuário vazado e mascarado disparando, ou não, as mesmas três categorias.

6. Taint analysis

taint.ts define um tipo Tainted: o valor carrega de onde veio e se já foi tratado. taint() marca a origem, propagate() mantém a marca numa transformação, sanitize() só libera o valor depois de aplicar a máscara do motor de PII.

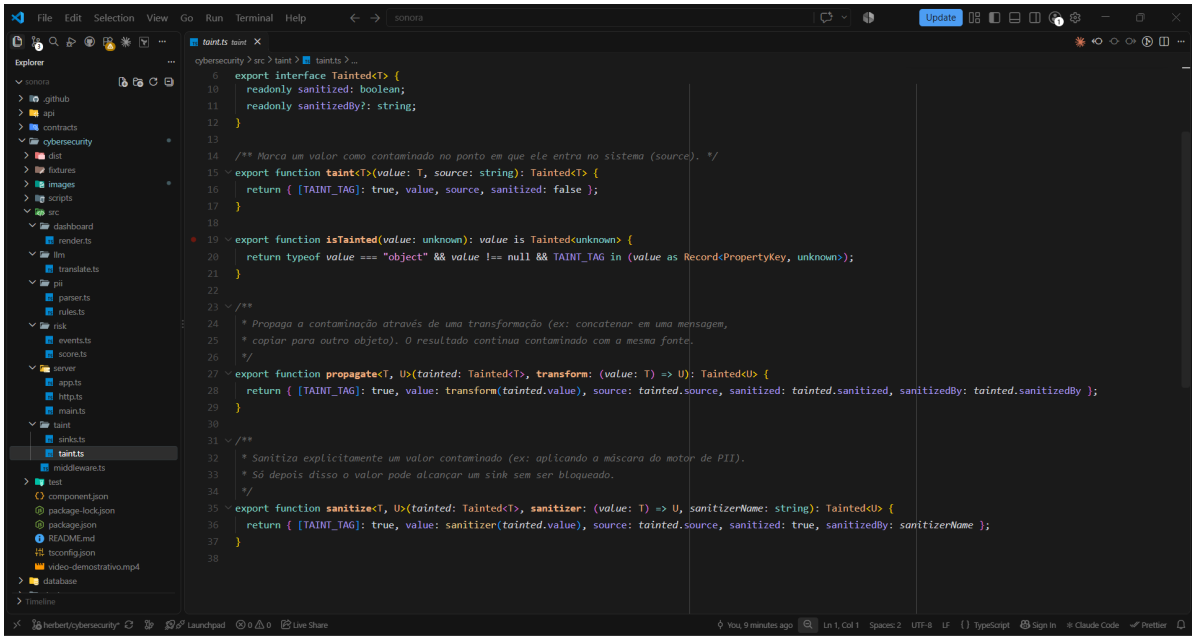


Figura 2 — src/taint/taint.ts: taint(), isTainted(), propagate() e sanitize()

Os sinks (sinks.ts) são o que prova a cadeia fonte -> propagação -> sink -> sanitização: safeLog() e safeHttpResponse() recusam qualquer valor marcado que não foi sanitizado. O middleware usa isso para mostrar, lado a lado, o mesmo campo passando pelo caminho inseguro (bloqueado) e pelo caminho seguro (permitido).

7. Tradução via IA

translate.ts recebe só o evento já sanitizado — o tipo de bloqueio, o sink, a categoria e a fonte — e pede para a IA traduzir isso em até duas frases simples, sem termos técnicos.

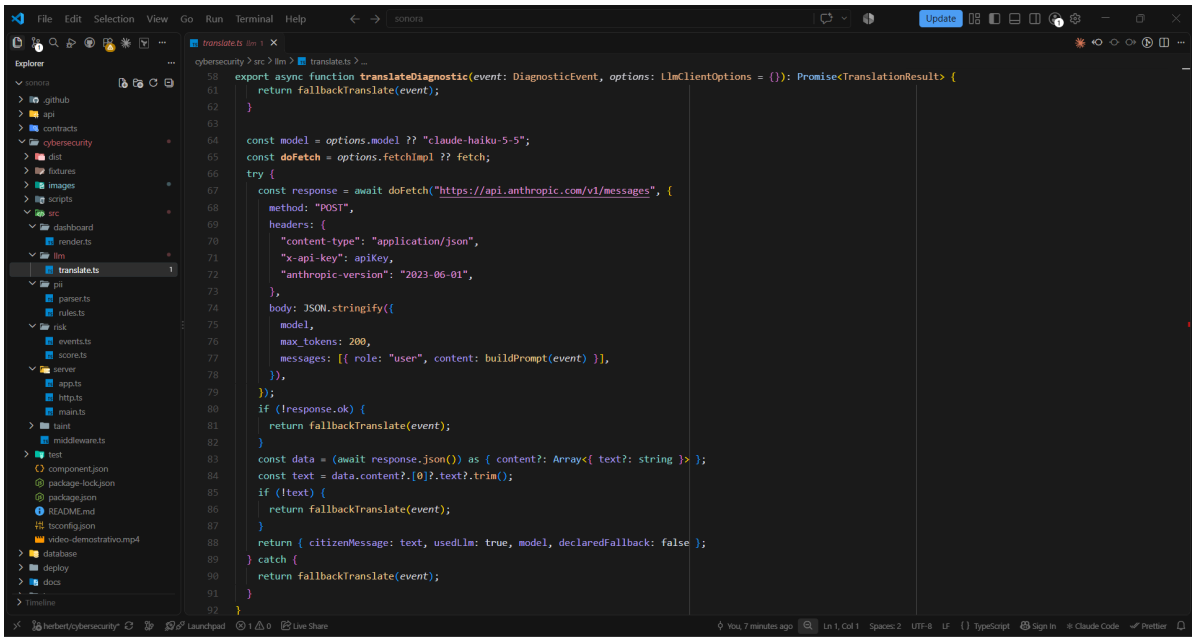
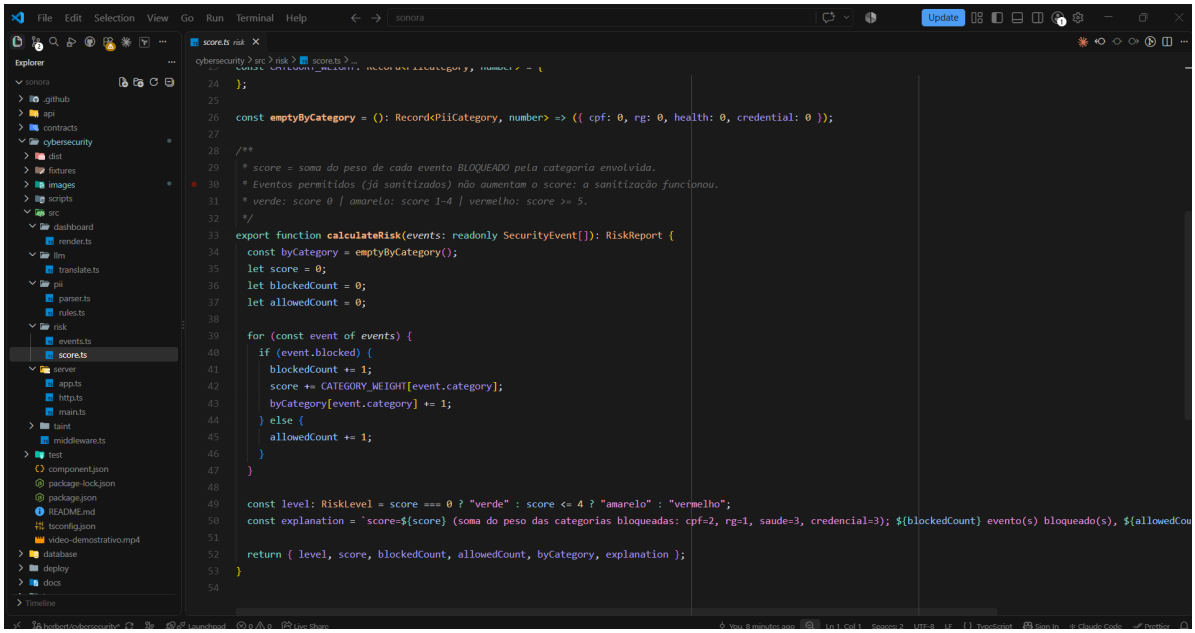


Figura 3 — src/llm/translate.ts: chamada à API da Anthropic, com fallback

Sem a chave da Anthropic configurada, a resposta vem de um texto fixo (o padrão deste repositório). Com a chave, a chamada acontece de verdade, como na imagem acima; se falhar, cai para o texto fixo em vez de quebrar o fluxo.

8. Dashboard Risco Cidadão

score.ts soma o peso de cada evento bloqueado pela categoria (CPF=2, RG=1, saúde=3, credencial=3). Eventos permitidos não somam nada — a sanitização funcionou. 0 é verde, até 4 é amarelo, 5 ou mais é vermelho.



```ts
};

const emptyByCategory = (): Record<PiiCategory, number> => ({ cpf: 0, rg: 0, health: 0, credential: 0 });

/**
 * score = soma do peso de cada evento BLOQUEADO pela categoria envolvida.
 * Eventos permitidos (já sanitizados) não aumentam o score: a sanitização funcionou.
 * verde: score 0 | amarelo: score 1-4 | vermelho: score >= 5.
 */
export function calculateRisk(events: readonly SecurityEvent[]): RiskReport {
  const byCategory = emptyByCategory();
  let score = 0;
  let blockedCount = 0;
  let allowedCount = 0;

  for (const event of events) {
    if (event.blocked) {
      blockedCount += 1;
      score += CATEGORY_WEIGHT[event.category];
      byCategory[event.category] += 1;
    } else {
      allowedCount += 1;
    }
  }

  const level: RiskLevel = score === 0 ? "verde" : score <= 4 ? "amarelo" : "vermelho";
  const explanation = `score=${score} (soma do peso das categorias bloqueadas: cpf=2, rg=1, saude=3, credencial=3); ${blockedCount} evento(s) bloqueado(s), ${allowedCou

  return { level, score, blockedCount, allowedCount, byCategory, explanation };
}
```

Figura 4 — src/risk/score.ts: calculateRisk() e a fórmula do farol

O painel (GET /dashboard) tem duas colunas: à esquerda, um campo livre para testar qualquer JSON, com botões que só preenchem exemplos prontos; à direita, a lista de eventos reais, mais recente primeiro. Um valor aleatório sem PII não muda nada; qualquer valor que bata numa regra move o farol na hora.

9. Vídeo demonstrativo

O vídeo mostra o painel em uso: cada um dos quatro exemplos é testado, um de cada vez, e dá para ver a tabela de eventos e o farol reagindo na hora — inclusive o contraste entre o exemplo vazado (dispara eventos) e o mascarado (não dispara nada).

Arquivo entregue junto do código: video-demostrativo.mp4.

10. Equipe

Equipe CitizenTech, a mesma em todas as disciplinas da Global Solutions:

- Leonardo Correa de Mello — RM 555573
- Felipe Soares Xavier — RM 556931
- Pedro Visconti Guidotte — RM 556630
- Herbert de Sousa Vilela — RM 555701
- Gabriel Figueira Flora — RM 556476

Trabalhamos juntos em todas as entregas da Global Solutions, incluindo esta.

11. Limites

- Ferramenta de auditoria com exemplos sintéticos — não varre o código real de Java, C# ou Mobile.
- O farol é ilustrativo da auditoria local, não uma classificação oficial de risco.
- Sem autenticação própria e log só em memória, por processo.
- Nenhum dado real é usado — CPF, RG e dados de saúde são sintéticos.